

Structural Design Patterns

Coding Exam – Practice Question Bank (modelled on the CSE-214 Online-2 test)

These practice questions follow the exact same layout as your real exam paper: a question code (A1, A2, B1 ...), the label "Structural Design Pattern", a suggested duration, a business scenario, and a one-line task. For each one you must (1) identify the correct structural pattern, and (2) implement a minimal working demonstration in Java. An answer key with the intended pattern for each question is given at the end of this document; full worked code for every question is in the companion "Process and Solutions" PDF.

CSE-214 Online – 2 (A1)

Structural Design Pattern

Duration: 20 minutes

You are building the alerting module for a Smart Parking Garage system. Whenever a monitored event occurs (a spot becomes free, a vehicle overstays its slot, an unauthorized vehicle is detected), the system dispatches a notification to the garage operator's tablet. Three basic delivery types are already implemented: Email Notification, SMS Notification, and Push Notification. Each delivers the same alert text through a different channel.

The operator's dashboard now needs to let staff optionally enhance how a specific alert is handled, and any combination of the following should be selectable for a single alert:

- **Enable Encryption** – the message body must be encrypted before it is sent.
- **Escalation Tag** – the message should be prefixed with "[ESCALATED]" before delivery.
- **Audit Logging** – the delivery must be recorded to an on-device audit log, including a timestamp, in addition to being sent normally.
- **Retry-on-Failure** – if delivery fails, the same alert should automatically be re-sent up to two more times, each attempt logged to the console.

Staff should be able to combine these enhancements freely and in any order for any of the three delivery types, and the operations team wants to be able to add further enhancements later without modifying the existing notification classes.

Task: Choose the appropriate design pattern for sending notifications so the app can seamlessly provide the enhancement options above, then implement a minimal demonstration showing at least two enhancements combined on top of one delivery type.

CSE-214 Online – 2 (A2)

Structural Design Pattern

Duration: 25 minutes

PetroFresh is a subscription-based cooking-gas and pantry-refill service. Customers set up a recurring monthly "Home Refill Bundle" that is delivered automatically. The platform sells preset bundles (Starter, Household, and Restaurant), each made up of multiple single items, as well as single items on their own (e.g., a gas cylinder, a bottle of cooking oil, a sack of rice). Every single item has a name, unit price, and weight.

Customers can build or edit their own Custom Refill Bundle by combining one or more preset bundles and/or single items. A custom bundle may contain only single items, only preset bundles, only other previously-created custom bundles, or any mixture of the three – and a custom bundle may be nested inside another custom bundle to an arbitrary depth.

The system must be able to calculate the total price and total weight of any configuration (a single item, a preset bundle, or an arbitrarily nested custom bundle), and must be able to print the complete structure of a custom bundle showing exactly what it contains, at every level of nesting.

Task: Choose the appropriate design pattern to solve this problem and implement a minimal demonstration, including at least one custom bundle nested inside another custom bundle.

CSE-214 Online – 2 (B1)

Structural Design Pattern

Duration: 25 minutes

MedLink is a telemedicine platform that has grown rapidly. It now needs to keep patients informed about key appointment events. Whenever a patient's appointment is booked, rescheduled, or a doctor starts the video call, the system must send a relevant notification to the concerned patient.

However, its patient base is diverse: some prefer Email, others rely on SMS, and a growing segment expects Push Notifications or WhatsApp messages. Product management has confirmed that new communication channels will keep being added as the platform expands into new regions. On the other side, the notifications themselves are also evolving — the content, tone, and required fields of an "appointment booked" message are very different from a "doctor is calling now" alert, and the clinical team regularly wants to introduce new event types as new features roll out.

The engineering team has been tasked with designing a notification system that stays clean and maintainable, and is easy to extend, without the codebase turning into an unmanageable mess every time a new channel or a new event type is introduced.

Task: Choose the appropriate design pattern to solve this problem and implement a minimal demonstration with at least two event types, each sent over at least two different channels.

CSE-214 Online – 2 (B2)

Structural Design Pattern

Duration: 20 minutes

A company is developing a Universal Remote Control App that lets users operate various home appliances from a single mobile screen. The system already supports appliances that follow the company's standard interface:

```
public interface ApplianceControl {  
    void switchOn();  
    void switchOff();  
}
```

Currently supported appliances include SmartOven, SmartWasher, and SmartFridge. All of them correctly

The company now wants to integrate third-party appliances from other manufacturers. However, these d

1. OldWaterHeater

```
public class OldWaterHeater {  
    public void activate() { }  
    public void deactivate() { }  
}
```

2. LegacyGarageDoor

```
public class LegacyGarageDoor {  
    public void open() { }  
    public void close() { }  
}
```

The app should continue to work using only the ApplianceControl interface, and the system should all

Task: Choose the appropriate design pattern so that all third-party appliances can be controlled through the ApplianceControl interface without modifying their original implementations, and implement a minimal demonstration controlling all appliances (standard and third-party) through that one interface.

CSE-214 Online – 2 (C1)

Structural Design Pattern

Duration: 25 minutes

PetroFresh traditionally offers three fixed Winter Care Packages every year: Basic, Comfort, and Deluxe. These packages have predefined contents and pricing and have become very popular; many customers not only buy them for personal use but also send them as gifts to relatives during the winter season. The structure and pricing of these packages are well-established and are part of PetroFresh's seasonal business model.

Recently, PetroFresh has received requests for slight customization. Some customers want to add a Blanket Pack, others want to include a Hot-Beverage Pack, and others want premium gift wrapping when sending a package to a loved one. Instead of allowing full customization, PetroFresh decided to provide limited flexibility by allowing customers to optionally add, in any combination: (1) a predefined Blanket Pack, (2) a predefined Hot-Beverage Pack, and (3) premium gift wrapping for presentation. [The Blanket Pack and Hot-Beverage Pack are also sold year-round as part of PetroFresh's normal business.]

Your task is to make the existing system capable of handling this winter season's flexibility without modifying the existing packages. You may assume the system already supports the previous business model. The system must correctly calculate the final price and description of the customized bundle while preserving the original package structure.

Task: Choose the appropriate design pattern and implement a minimal demonstration in which at least two optional add-ons are applied to one of the fixed Winter Care Packages.

CSE-214 Online – 2 (C2)

Structural Design Pattern

Duration: 25 minutes

PetroFresh offers multiple delivery options for its refill bundles:

- Standard Delivery (delivered within 24 hours)
- Express Delivery (delivered within 4 hours)
- Scheduled Delivery (delivered at a chosen time slot)

Each delivery type has its own pricing logic and estimated delivery time calculation. Regardless of type, every order must ultimately be dispatched using a physical transportation method. Until now, PetroFresh has relied on motorbike couriers and van deliveries. Recently, however, the company decided to introduce drone delivery in selected cities to speed up last-mile logistics for small, light refills. As a result, the development team now finds itself updating multiple classes just to support this one new transportation method — adding drone-specific dispatch logic, tracking integration, and safety checks in several places. The situation is becoming harder to manage because PetroFresh is also planning to introduce autonomous robot delivery for gated communities in the near future.

The challenge is that delivery-type business policies may change frequently, while transport technologies may expand over time, but the current design tightly couples everything, forcing code duplication and repeated modification whenever either dimension changes.

Task: Redesign the system with an appropriate design pattern so that new transport technologies (like drones or robots) can be introduced without rewriting existing delivery-type logic, and vice versa. Implement a minimal demonstration covering at least two delivery types, each dispatched through at least two different transport technologies.

CSE-214 Online – 2 (D1)

Structural Design Pattern

Duration: 25 minutes

SecureVault is an internal document-sharing tool used by a company's finance department. Some financial reports are very large PDF files, and generating an in-app preview for a report is computationally expensive (it involves rendering thumbnails for every page). Reports should only ever be rendered the first time someone actually opens them, and the rendered result should be reused for all later views by any user, rather than re-rendered every time.

In addition, every report carries a required clearance level (1 to 5). A staff member whose clearance is below a report's required level must be refused access immediately, and – importantly – the expensive rendering step must never even be attempted for a user who is not allowed to see the report.

The rest of the application should not need to know or care whether it is dealing with a report that has already been rendered, one that needs rendering right now, or one that will be denied outright; it should always call the same single method to try to view a report.

Task: Choose the appropriate design pattern and implement a minimal demonstration showing: (a) a low-clearance user being denied without triggering any rendering, (b) a sufficiently-cleared user triggering the first (expensive) render, and (c) a second, different sufficiently-cleared user viewing the same report afterward with no re-render.

CSE-214 Online – 2 (D2)

Structural Design Pattern

Duration: 20 minutes

GreenStay is a smart-apartment platform. Each apartment exposes several independent subsystems with fairly detailed APIs of their own: a Climate Control subsystem (separate methods to set temperature and set fan speed), a Lighting subsystem (separate methods to dim each of three rooms), a Security subsystem (separate methods to arm/disarm and to lock/unlock the main door), and an Energy Monitor subsystem (a method to switch between "Normal" and "Eco" power profiles).

Whenever a tenant leaves the apartment, six to eight of these individual calls must happen in a specific order (dim every room to 0%, set a lower temperature, switch the energy profile to Eco, lock the door, and arm security) — and a mirrored, differently-ordered sequence must happen when the tenant returns home. Support staff found that tenants kept getting the order wrong when scripting their own routines directly against the subsystem APIs, occasionally arming security before the door was locked.

Task: Choose the appropriate design pattern to expose two simple operations – leaving home and arriving home – that internally coordinate all four subsystems in the correct order, while leaving each subsystem's own detailed API untouched and still directly usable by advanced users. Implement a minimal demonstration of both operations.

Answer Key

A1: Decorator | A2: Composite | B1: Bridge | B2: Adapter | C1: Decorator | C2: Bridge |
D1: Proxy | D2: Facade

Note: A1–C2 mirror the pattern distribution of your real CSE-214 Online-2 exam (Decorator, Composite, Bridge, Adapter, Decorator, Bridge). D1 and D2 are added as extra practice in case Proxy or Facade also appear, since the syllabus lists several structural pattern methods.